



SULTAN QABOOS UNIVERSITY

FYP Online Grading Platform

Complete Functional, Operational, and Migration Guide

From legacy Excel and MATLAB grading to a secure, role-based digital workflow

CURRENT DEMONSTRATION

Azure for Students | Docker | HTTPS | PostgreSQL | Spring Boot | React

Prepared for project delivery and migration to the university server

August 2026

Realized by Ranyam Mejri

Guide Overview

This guide documents the delivered platform, its academic workflows, the current Azure environment, daily operation, backup and recovery, and the complete transition to SQU infrastructure.

- 01 Project Purpose
- 02 Work Completed
- 03 Actors and Responsibilities
- 04 Main Functional Modules
- 05 Technical Architecture
- 06 Current Azure Demonstration Environment
- 07 Using the Application on Azure
- 08 Backup and Restore
- 09 Migration to the University Server
- 10 Security and Operational Rules
- 11 Useful Verification Commands
- 12 Current Limitations and Required Institutional Actions
- 13 Handover Summary

1. Project Purpose

The FYP Online Grading Platform replaces the former Final Year Project process based on separate evaluator Excel workbooks and a MATLAB consolidation program.

In the former process:

1. Academic staff used different multi-sheet Excel files for each track and assessment.
2. Evaluators entered scores manually in their own copies.
3. Files were collected from multiple colleagues.
4. A MATLAB program read the submitted files.
5. MATLAB averaged evaluator marks, applied the assessment weights, and generated `Final_Evaluation_Summary.xlsx`.
6. Missing files, incomplete sheets, formula differences, and manual handling could affect traceability and reliability.

The platform centralizes the same academic process:

1. The administrator imports the annual academic data.
2. The system creates projects, teams, students, actors, phases, and assignments.
3. Each evaluator sees only the projects assigned to that account.
4. Scores are saved as drafts while work is in progress.
5. Only validated and locked forms contribute to official grades.
6. The backend averages repeated evaluator contributions and applies configured weights.
7. Authorized users review, calculate, publish, export, and audit the results.

2. Work Completed

The following work has been completed during the project:

- the original specification and legacy Excel/MATLAB process were analyzed;
- the backend was implemented with Java 21, Spring Boot, Spring Security, JPA, PostgreSQL, Flyway, Spring Mail, and Swagger/OpenAPI;
- the React frontend was redesigned as an English-only, responsive, role-based interface;
- student records were separated from platform user accounts;
- dedicated roles and dashboards were added for administrators, coordinators, supervisors, report evaluators, faculty evaluators, and Industry Guests;
- annual Excel initialization and validation workflows were implemented;
- project, team, student, assignment, phase, deadline, rubric, grading, notification, reporting, and audit workflows were implemented;
- report evaluation and Industry Demo Day rules were added from the supplied legacy forms;
- the MATLAB consolidation behavior was replaced by backend calculation and Excel export services;
- draft, validation, locking, deadline, reminder, and extension workflows were implemented;
- local Docker startup and a production Docker deployment were prepared;
- GitHub Actions now builds production backend and frontend container images;
- an Azure for Students VM was created and secured;
- Caddy and Let's Encrypt provide automatic HTTPS;
- a monthly Azure budget alert was configured;

- the live application and administrator authentication were verified.

3. Actors and Responsibilities

3.1 Administrator

The administrator configures and controls the platform. Main actions include:

- import and validate annual initialization workbooks;
- manage platform users and roles;
- manage student academic records;
- manage tracks, projects, teams, and team membership;
- assign one or two supervisors to a project;
- assign report evaluators, faculty evaluators, and Industry Guests;
- create and manage FYP I, FYP II, and Demo Day phases;
- define opening dates, deadlines, status, and extensions;
- configure evaluation forms, rubric criteria, and grade rules;
- monitor drafts, locked forms, missing evaluations, and completion;
- calculate, recalculate, publish, and export grades;
- approve or reject deadline-extension requests;
- generate and e-mail reports;
- inspect notifications and audit history.

3.2 FYP Coordinator

The coordinator supervises the academic process without receiving unrestricted system administration rights. The coordinator can monitor progress, view authorized results, inspect completion, and access permitted reports and exports.

3.3 Supervisor

A supervisor sees only supervised projects and can:

- inspect assigned project and team information;
- complete the Supervisor FYP I and Supervisor FYP II forms;
- save evaluation work as a draft;
- validate and lock a complete form before the deadline;
- view relevant notifications and deadlines;
- request an extension without choosing a new deadline.

3.4 Report Evaluator

A report evaluator evaluates the students' written Report I and Report II using the official paper-report rubric. The evaluator sees only assigned projects and authorized report forms.

3.5 Faculty Evaluator

A faculty evaluator grades assigned oral presentations for FYP I and FYP II. The account cannot access unassigned projects or unauthorized assessment types.

3.6 Industry Guest

An Industry Guest is an external, invitation-based actor. The account:

- is created or imported by the administrator;
- receives a temporary invitation and activation workflow;
- has a mandatory access-expiration date;
- sees only assigned Demo Day projects;
- can complete only the Industry Demo Day form;
- cannot edit FYP I, FYP II, supervisor, report, or oral evaluations;
- may view authorized results from other phases without modifying them.

3.7 Students

Students are academic data records, not platform actors. They have no login account or dashboard. Student records are linked to teams, projects, tracks, cohorts, evaluations, and final grades.

4. Main Functional Modules

4.1 Authentication and Access Control

- role-based authentication and dashboard redirection;
- separate authorization scope for every actor;
- no public self-registration for university staff;
- configurable SQU OpenID Connect single sign-on;
- temporary Industry Guest invitation and activation;
- forgot-password and reset-password workflow;
- token-based authenticated API access;
- backend authorization against direct access to unassigned projects;
- configurable expiration for external access.

The current public demonstration temporarily uses local login because SQU identity-provider credentials have not yet been supplied. The university deployment should disable local internal login and enable SQU SSO.

4.2 Annual Excel Initialization

The administrator can initialize the academic year from one master workbook. Supported sheets include:

- STUDENTS;
- ADMINISTRATORS;
- COORDINATORS;
- SUPERVISORS;
- REPORT_EVALUATORS;
- FACULTY_EVALUATORS;
- INDUSTRY_GUESTS;
- PHASES;
- PROJECT_ASSIGNMENTS.

The import process supports:

- analysis without saving;
- sheet, row, field, and reference validation;
- transactional initialization;
- stable identifiers and idempotent updates;
- one to five students per project;
- one or two supervisors per project;
- multiple evaluator assignments;
- project, team, student, actor, track, and phase relationships;
- fictional demonstration templates and separate official student updates.

4.3 Academic Data Management

Searchable and paginated CRUD interfaces are available for users, students, tracks, projects, teams, assignments, phases, forms, criteria, grading rules, notifications, reports, and audit records. Destructive operations require confirmation.

4.4 Phases, Deadlines, and Extensions

The administrator can create, edit, open, close, archive, and filter phases. Each phase contains a type, sequence, start date, deadline, cohort or year, and status.

When a deadline expires:

- new final validation is rejected;
- unfinished drafts remain excluded from official calculations;
- the evaluator may send an extension request and reason;
- only the administrator can choose the new deadline;
- the administrator approves or rejects the request;
- the requester receives an in-app and e-mail notification.

The platform creates reminders approximately 24 hours and 12 hours before a deadline.

4.5 Evaluation Lifecycle

Every evaluation follows the same controlled lifecycle:

1. The system checks the authenticated role.
2. It checks the evaluator-project assignment.
3. It checks the permitted assessment type.
4. It checks that the phase and personal deadline allow submission.
5. The evaluator enters criterion scores and comments.
6. Changes are created or updated as a draft.
7. The evaluator completes all mandatory criteria.
8. The evaluator confirms final validation.
9. The submission is locked.
10. Only the locked submission becomes eligible for consolidation.

Supported assessment types are Supervisor FYP I, Report FYP I, Oral FYP I, Supervisor FYP II, Report FYP II, Oral FYP II, and Industry Demo Day.

4.6 Grade Calculation and MATLAB Replacement

The Spring Boot calculation engine reads locked submissions from PostgreSQL instead of collecting evaluator workbooks. It groups results by project, student, phase, evaluator, and assessment type.

Important rules include:

- valid zero scores remain valid values;
- missing scores are not silently converted to zero;
- multiple locked evaluator results for one assessment type are averaged;
- drafts are excluded;
- expired but unvalidated forms are excluded;
- supervisor results may be individual to each student;
- report results use the official weighted criterion structure;
- Industry Demo Day uses the official 2 / 1 / 4 / 2 / 1 weighting;
- administrator-configured phase percentages are applied after normalization.

Default FYP I weights are Supervisor 40%, Report 35%, and Oral 25%. Default FYP II weights are Supervisor 30%, Report 25%, Oral 25%, and Industry Demo Day 20%.

The administrator can inspect missing forms, calculate or recalculate results, review them, and publish approved grades.

4.7 Reports and Exports

The platform can generate project, phase, and final reports. Exported workbooks can contain:

- `LEGACY_SUMMARY` for compatibility with the former process;
- `FINAL_SUMMARY` for consolidated student results;
- `EVALUATOR_DETAILS` for locked evaluator contributions;
- `MISSING_FORMS` for incomplete assessment tracking;
- `AUDIT_TRAIL` for sensitive workflow events.

Generated reports are persisted in a Docker volume and can be downloaded, regenerated, e-mailed, or deleted by authorized users.

4.8 Notifications, E-mail, and Audit

All actors have a notification bell and contextual notification window. Notifications can redirect users to the related phase, evaluation, extension request, or report.

E-mail events include Industry invitations, password resets, deadline reminders, extension decisions, and generated reports. Audit records preserve sensitive administrative and grading actions.

5. Technical Architecture

5.1 Application Stack

- **Frontend:** React, TypeScript, Vite, responsive role-based interface;
- **Backend:** Java 21 LTS and Spring Boot 3.5;
- **Database:** PostgreSQL 18;
- **Database migrations:** Flyway;
- **API documentation:** Swagger/OpenAPI;

- **E-mail:** Spring Mail through SMTP;
- **Containers:** Docker and Docker Compose;
- **Reverse proxy and TLS:** Caddy;
- **CI/CD build:** GitHub Actions;
- **Container registry:** GitHub Container Registry.

The deployed solution uses separate runtime containers for PostgreSQL, Mailpit, backend, frontend, and Caddy. The current Spring Boot backend is one deployable backend service organized by functional modules; it is not a set of independently deployed domain microservices.

5.2 Request Flow



PostgreSQL port 5432, backend port 8080, Mailpit ports 1025/8025, and Caddy administration port are not exposed publicly.

6. Current Azure Demonstration Environment

6.1 Azure Resources

- subscription: Azure for Students;
- resource group: rg-fyp-grading-demo;
- VM: vm-fyp-grading-demo;
- region: Spain Central;
- size: Standard_B2ats_v2;
- public IP: 68.221.71.117;
- public URL: <https://squ-fyp-grading.68-221-71-117.sslip.io>;
- monthly budget monitor: fyp-demo-budget, amount USD 5;
- operating system: Ubuntu Linux;
- swap: 4 GB;
- application directory: /opt/fyp-platform.

The VM size is eligible for the Azure free allowance, subject to the current Azure for Students offer and quota. The public IP and other resources remain metered Azure resources and may consume student credit. Budget alerts monitor cost but do not automatically stop the VM.

6.2 Current Authentication State

- local internal login is temporarily enabled for the public demonstration;
- SQU SSO is not yet configured;
- the administrator bootstrap credential is stored only on the VM;

- no credential is stored in Git or in this document;
- only fictional data should be imported into the public environment.

The administrator should change the temporary password after first login. When SQU SSO becomes available, set `LOCAL_INTERNAL_LOGIN_ENABLED=false` and configure the SQU OIDC variables.

6.3 Current E-mail State

The Azure demonstration uses an internal Mailpit container. The backend performs SMTP operations and Mailpit captures messages, but they are not delivered to real external recipients. Mailpit is not exposed to the public Internet.

For real delivery, replace the Mailpit variables with authorized SQU SMTP or SendGrid SMTP settings.

7. Using the Application on Azure

7.1 Normal User Access

1. Open `https://squ-fyp-grading.68-221-71-117.sslip.io`.
2. Enter the provisioned account e-mail and password.
3. The backend verifies the account, status, role, and expiration.
4. The frontend redirects the user to the appropriate dashboard.
5. The user sees only authorized projects and actions.

7.2 Administrator First Use

1. Sign in with the temporary administrator account supplied separately.
2. Change the temporary password.
3. Open Excel Imports.
4. Upload the fictional demonstration workbook.
5. Run analysis without saving.
6. Correct any reported sheet, row, field, or reference errors.
7. Run the transactional initialization.
8. Review users, students, projects, teams, assignments, and phases.
9. Verify deadlines and evaluation forms.
10. Sign out and test every actor with only the assigned data.

7.3 Connecting to the VM

From the authorized administrator computer:

```
ssh azureuser@fyp-squ-ranym-89baa6.spaincentral.cloudapp.azure.com
```

SSH is restricted to the administrator's current public IP. If that IP changes, update the Azure NSG rule before reconnecting.

7.4 Check Service Status

```
cd /opt/fyp-platform
docker compose --env-file .env.production -f compose.production.yaml ps
```

All five services should be running. PostgreSQL, backend, and frontend should report `healthy`.

7.5 Read Logs

```
cd /opt/fyp-platform
docker compose --env-file .env.production -f compose.production.yaml logs --tail=200 backend
docker compose --env-file .env.production -f compose.production.yaml logs --tail=200 frontend
docker compose --env-file .env.production -f compose.production.yaml logs --tail=200 caddy
```

Follow logs live with `-f`, then stop viewing with `Ctrl+C`.

7.6 Restart the Platform

```
cd /opt/fyp-platform
docker compose --env-file .env.production -f compose.production.yaml restart
```

7.7 Deploy a New Version

1. Push tested changes to `main`.
2. Wait for the GitHub Actions workflow **Publish production containers** to succeed.
3. Connect to the VM.
4. Run:

```
cd /opt/fyp-platform
git pull --ff-only
./deploy/azure/deploy.sh
```

The deployment script pulls the current container images, recreates changed services, and displays their status.

7.8 Stop and Start the Azure VM

From a computer authenticated with Azure CLI:

```
az vm deallocate --resource-group rg-fyp-grading-demo --name vm-fyp-grading-demo
az vm start --resource-group rg-fyp-grading-demo --name vm-fyp-grading-demo
```

Use **deallocate**, not only operating-system shutdown, when the VM should stop consuming compute allocation. Some attached resources can remain billable.

8. Backup and Restore

8.1 Database Backup

Run before every important update or migration:

```
cd /opt/fyp-platform
mkdir -p backups
docker compose --env-file .env.production -f compose.production.yaml \
  exec -T postgres pg_dump \
  -U fyp_platform -d fyp_grading_platform \
  > "backups/fyp-$(date +%Y%m%d-%H%M%S).sql"
```

Copy the backup to a protected location outside the VM. A backup stored only on the same VM is not sufficient disaster recovery.

8.2 Generated Reports

Generated files are stored in the Docker `report_data` volume. Back up this volume or copy the required generated reports before server migration.

8.3 Database Restore

On the target server, after starting PostgreSQL:

```
cd /opt/fyp-platform
docker compose --env-file .env.production -f compose.production.yaml \
  exec -T postgres psql \
  -U fyp_platform -d fyp_grading_platform \
  < /secure-backups/fyp-backup.sql
```

Restore into a clean, compatible database and verify Flyway state before opening the service to users.

9. Migration to the University Server

9.1 Information Required from SQU

The university IT or identity team must provide:

- Linux server or virtual machine specifications;
- official DNS name, for example `fyp-grading.squ.edu.om`;
- firewall and reverse-proxy rules;
- TLS certificate policy, or permission to use Let's Encrypt;
- SQU OpenID Connect issuer URL;
- OIDC client ID and client secret;
- approved callback and logout URLs;
- required OIDC scopes and e-mail claim;
- SQU SMTP hostname and port, or approved SendGrid account;
- SMTP username, password/API key, sender address, TLS, and relay policy;
- backup location and retention requirements;
- monitoring, log-retention, privacy, and security requirements.

9.2 Target Server Requirements

Recommended minimum for normal institutional use:

- Ubuntu Server LTS or another supported Linux distribution;
- at least 2 CPU cores;
- at least 4 GB RAM;
- at least 40 GB persistent storage, adjusted for reports and backups;
- Docker Engine and Docker Compose plugin;
- outbound HTTPS access for image pulls and certificate renewal;
- outbound SMTP access to the approved mail service;
- inbound TCP 80 and 443;
- restricted SSH administration;
- regular operating-system security updates.

For higher availability, the university may replace the local PostgreSQL container with a managed or centrally administered PostgreSQL service and use institutional backup infrastructure.

9.3 Pre-Migration Preparation

1. Freeze administrative changes during the final backup window.
2. Confirm all important evaluation forms are saved and locked as expected.
3. Create a final PostgreSQL backup.
4. Back up generated reports.
5. record the current image tag or Git commit.
6. Prepare a new production `.env.production` file without copying obsolete demonstration settings.

7. Store all secrets in the university-approved secret manager or protected server file.

9.4 Install the Application on the University Server

```
sudo install -d -o "$USER" -g "$USER" /opt/fyp-platform
git clone https://github.com/ranym-eng/FYP-Online-Grading-Platform.git /opt/fyp-platform
cd /opt/fyp-platform
cp .env.production.example .env.production
chmod 600 .env.production
```

Edit `.env.production` with the official domain, strong unique secrets, database settings, SMTP settings, and SQU SSO settings.

9.5 Configure the Official Domain and HTTPS

1. Create an A or AAAA DNS record from the official SQU hostname to the server.
2. Open only ports 80 and 443 to public users.
3. Restrict SSH to authorized administration networks or VPN.
4. Keep PostgreSQL, backend, SMTP-capture, and Caddy administration ports private.
5. Set `APP_DOMAIN` to the official hostname.
6. Set `ACME_EMAIL` to an institutional operations address.
7. If SQU supplies its own certificate, adapt Caddy to use the approved certificate and private key instead of ACME.

9.6 Configure SQU SSO

Set the production environment values:

```
LOCAL_INTERNAL_LOGIN_ENABLED=false
SQU_SSO_ENABLED=true
SQU_SSO_CLIENT_ID=<provided-by-squ>
SQU_SSO_CLIENT_SECRET=<stored-secret>
SQU_SSO_ISSUER_URI=<official-oidc-issuer>
SQU_SSO_SCOPES=openid,profile,email
SQU_SSO_EMAIL_CLAIM=email
SQU_SSO_ALLOWED_DOMAIN=squ.edu.om
```

The identity provider must authorize the callback URLs used by the application. Imported internal users must have e-mail addresses matching the identities returned by SQU SSO.

After SSO validation:

- confirm administrators reach the administrator dashboard;
- confirm supervisors and evaluators reach only their dashboards;
- confirm unknown SQU users are denied access;
- confirm Industry Guest invitation access remains separated from internal SSO;
- remove or securely archive temporary local demonstration credentials.

9.7 Configure Real E-mail Delivery

For SQU SMTP, use values similar to:

```
MAIL_HOST=<squ-smtp-host>
MAIL_PORT=587
MAIL_USERNAME=<approved-account>
MAIL_PASSWORD=<stored-secret>
MAIL_FROM=<approved-fyp-address>
MAIL_SMTP_AUTH=true
MAIL_STARTTLS_ENABLE=true
MAIL_STARTTLS_REQUIRED=true
```

SendGrid can be used through its SMTP interface with `smtp.sendgrid.net`, port 587, username `apikey`, and the API key stored as `MAIL_PASSWORD`.

Test invitation, password reset, deadline reminder, extension decision, and report-delivery messages before production approval. Remove the Mailpit service from the institutional production composition if it is no longer needed.

9.8 Restore Data and Reports

1. Start only the target PostgreSQL service.
2. Restore the final SQL backup.
3. Restore or copy generated reports.
4. Start the backend and verify Flyway migrations.
5. Start the frontend and Caddy.
6. Verify health checks and logs.
7. Test the system with designated non-production SQU accounts.

9.9 Acceptance Tests Before Handover

The university should validate:

- HTTPS certificate and official domain;
- SQU SSO login and role redirection;
- Industry Guest invitation and expiration;
- annual workbook preview, validation, and initialization;
- project and team relationships;
- one-to-five-student and one-to-two-supervisor constraints;
- role-scoped project visibility;
- every evaluation form type;
- draft saving and locked final validation;
- closed and expired deadline behavior;
- extension request and administrator decision;
- 24-hour and 12-hour reminders;
- grade normalization, averaging, weighting, and publication;
- MATLAB-compatible and enhanced Excel exports;
- real SMTP delivery;
- notifications and contextual navigation;
- audit history;
- backup and restore;
- desktop, tablet, and mobile layouts;
- denied access to ports 5432, 8080, 8025, and 1025 from the Internet.

9.10 Final Cutover

1. Announce a maintenance window.
2. Stop new changes on the Azure demonstration.
3. Take the final database and report backup.
4. Restore the final data on the university server.
5. Run the acceptance tests.

6. Change the official DNS to the university server.
7. Monitor authentication, e-mail, logs, and calculation workflows.
8. Keep Azure temporarily available only as a rollback environment if approved.
9. Deallocate and then delete the Azure demonstration resources after formal acceptance and backup verification.

10. Security and Operational Rules

- never commit `.env.production`, private keys, SMTP credentials, OIDC secrets, or database passwords;
- rotate the temporary administrator password;
- disable local internal login when SQU SSO is ready;
- use only fictional data on the public demonstration VM;
- restrict SSH by IP or institutional VPN;
- expose only HTTP/HTTPS publicly;
- apply operating-system and container updates;
- back up PostgreSQL and reports outside the application server;
- review Azure credit and budget notifications;
- test restoration, not only backup creation;
- preserve audit records according to institutional policy;
- validate calculations after rubric or grade-rule changes;
- use separate demonstration, staging, and production environments where possible.

11. Useful Verification Commands

Public HTTPS

```
curl.exe -I https://squ-fyp-grading.68-221-71-117.sslip.io
```

Azure VM State

```
az vm show `
  --resource-group rg-fyp-grading-demo `
  --name vm-fyp-grading-demo `
  --show-details `
  --output table
```

Docker Service State

```
cd /opt/fyp-platform
docker compose --env-file .env.production -f compose.production.yaml ps
```

Backend Health from the Private Docker Network

```
docker compose --env-file .env.production -f compose.production.yaml \
  exec backend wget -q -O - http://localhost:8080/actuator/health
```

Resource Usage

```
free -h
df -h
docker stats --no-stream
docker system df
```

12. Current Limitations and Required Institutional Actions

The application is operational for demonstration, but institutional production still requires:

- official SQU OIDC/SSO credentials and callback registration;
- official SMTP or SendGrid credentials for real e-mail delivery;
- an official SQU domain;
- university-approved hosting, certificate, backup, monitoring, and privacy policies;
- final validation of assessment formulas by the academic owner;
- formal load, security, recovery, and user-acceptance testing;
- removal of fictional demonstration data before real academic use.

13. Handover Summary

The same application code, Docker images, database model, evaluation logic, and frontend can move from Azure to the university server. The main migration changes are infrastructure configuration:

- replace the temporary demonstration domain with the official SQU domain;
- replace Mailpit with SQU SMTP or approved SendGrid delivery;
- replace temporary local internal login with SQU SSO;
- restore the approved PostgreSQL backup and report files;
- apply university firewall, backup, monitoring, and security controls.

No rewrite of the grading engine or React application is required for this migration.